

Why EPIC Does Not Need Custom Build Steps

Background

A request has been raised to allow application teams to provide custom build steps within the EPIC pipeline. The rationale is that some teams may have unique build processes based on the intent or function of their applications.

This document explains why custom build steps are unnecessary, how common build scenarios are already handled, and why introducing them would undermine the core value of the platform.

The EPIC Principle

EPIC exists to eliminate pipeline fragmentation. Every application team gets the same pipeline. They don't build one, maintain one, or debug one. They provide a config file (`.pipeline/epic.json`) that tells EPIC what to build and where to deploy. That's it.

The moment we allow teams to inject arbitrary steps into the pipeline, we lose the consistency, auditability, and maintainability that EPIC was designed to provide. We would effectively be managing 30+ variations of a pipeline instead of one.

Why Build Steps Don't Vary by Application Intent

Building a .NET API that serves financial data is the same `dotnet publish` as building a .NET API that serves weather data. Building an Angular dashboard for fleet management is the same `ng build` as building an Angular dashboard for customer support.

The build process is determined by the **application type**, not the application's business purpose. EPIC already handles this through the `appType` field in `epic.json`, which dispatches to the correct build template:

appType	Build Process
angular	<code>npm install + ng build</code>
html	File copy (rsync)
dotnet	<code>dotnet restore + dotnet publish</code>
dotnet_framework	MSBuild
java	Maven or Gradle
php	Composer
python	<code>pip / setuptools</code>
ami	EC2 Image Builder

If an application uses .NET, it builds like .NET. If it uses Angular, it builds like Angular. There is no scenario where the business logic of the application changes how the compiler or bundler operates.

Common Misunderstandings

When teams request custom build steps, the actual need typically falls into one of three categories. None of them require changes to EPIC.

1. Pre-build or post-build tasks that belong in the application repo

Examples:

- Running a code generator before compilation
- Copying environment-specific config files
- Compiling SCSS or other preprocessor files
- Running database migration scripts as part of build output

Why this is not an EPIC concern:

Every modern build system provides hooks for pre-build and post-build tasks:

- **.NET** — MSBuild targets (`<Target Name="PreBuild" BeforeTargets="Build">`) or `dotnet tool` commands in the project file
- **Node / Angular** — `npm` scripts in `package.json` (`prebuild`, `postbuild`, or custom scripts called from the build script)
- **Java** — Maven phases or Gradle task dependencies
- **Python** — `setup.py` commands, `Makefile`, or `tox` configurations

These hooks execute automatically when EPIC runs the standard build command. EPIC does not need to know about them. The application team owns their build configuration inside their repository, and the pipeline simply invokes the standard build tool.

2. A new application type that EPIC doesn't support yet

Examples:

- A team building a Go microservice
- A team building a Rust CLI tool
- A team using a monorepo with multiple build outputs

Why this is an EPIC enhancement, not a custom step:

If a team is using a language or framework that EPIC does not yet support, the correct response is to add a new `appType` to the engine. This is a one-time addition to the platform that benefits every future team using that technology. It is not a reason to open an escape hatch for arbitrary build commands.

3. A misunderstanding of responsibility boundaries

Examples:

- “Our app needs to download a dependency from a private registry before building”
- “We need to set specific environment variables during the build”
- “Our build requires a specific version of Node that isn’t the default”

Why EPIC already handles this:

- **Private registries** — Handled via `.npmrc`, `nuget.config`, or `pip.conf` in the application repo. Authentication tokens are managed through the pipeline’s service connections, not custom steps.
- **Environment variables** — Build-time variables belong in the application’s build configuration, not the pipeline.
- **Runtime versions** — EPIC supports `runtimeVersion` in `epic.json`. Teams specify the version they need, and the pipeline installs it before building.

The Risk of Allowing Custom Build Steps

If EPIC allows custom build steps, even as an optional feature:

1. **It will become the default.** Teams will use the escape hatch instead of conforming to standards, because it is easier in the short term.
2. **Consistency disappears.** Security scans, compliance audits, and debugging all assume a known pipeline shape. Custom steps break that assumption.
3. **Support burden multiplies.** The EPIC team would need to troubleshoot arbitrary scripts they did not write and cannot control.

Recommendation

Do not add custom build steps to EPIC. Instead:

1. **Ask for concrete examples.** Request 2-3 specific scenarios from the teams requesting this feature. Identify which of the three categories above they fall into.
2. **Guide teams to use their build system’s native hooks** for pre-build and post-build tasks. This is the correct architectural boundary.
3. **Add new appType support** if a genuinely unsupported technology is identified. This strengthens EPIC for everyone.
4. **Document the boundary clearly.** EPIC owns the pipeline. The application team owns their build configuration. The `epic.json` contract is the interface between them.

The strength of EPIC is that it removes decisions, not that it adds options.